

DOING IT MANY TIMES

Loops and your first collection

ANATOMY OF A LOOP

```
+-----+
| starting state | friends starts empty
+-----+-----+
      |
      v
+-----+-----+
|  stop check   |<-----+ "did the user
| friendName empty?|      | press enter on
+-----+-----+      | an empty line?"
      no | yes      |
          | +-----> done      |
          v              |
+-----+-----+      |
|  loop body    | read a name,  |
|              | push_back it  |
+-----+-----+      |
          |              |
          +---- go back and check ----+
          |              |
```

WHILE

while - repeat until told to stop

```
while (true) {  
    std::print("Friend's name: ");  
    std::string friendName;  
    std::getline(std::cin, friendName);  
  
    if (friendName.empty()) {  
        break; // empty line means we're done  
    }  
  
    friends.push_back(friendName);  
}
```

BREAK

break

break exits a loop immediately, regardless of the loop's condition.

```
if (friendName.empty()) {  
    break;  
}
```

Without it, **while (true)** would never stop on its own.

A COLLECTION, READY-MADE

You don't have to build one

C++ ships with ready-made collections - you don't have to build your own growable list.

```
std::vector<std::string> friends;
```

`std::vector<std::string>` is a list of strings that grows as you add to it with `push_back`.

Same idea as `std::string` being a ready-made collection of characters. You don't need to know how `std::vector` works inside to use one.

ANATOMY OF A FOR LOOP

```
+-----+
| starting state | start at the first
+-----+-----+ friend in the list
      |
      v
+-----+
| current friend |<-----+
| in list?      |          |
+-----+-----+          |
      yes | no          |
      | +-----> done  |
      v                  |
+-----+-----+          |
| loop body    | print this |
|              | friend's name |
+-----+-----+          |
      |                  |
      +-- move to the next friend --+
```

LOOPING OVER IT

Looping over the collection

```
std::println("\nYou added {} friend(s)", friends.size());  
for (const std::string& friendName : friends) {  
    std::println(" - {}", friendName);  
}
```

A range-based **for** visits every element, one at a time - no manual indexing required. It's another variant of loops in C++.

DEBUG THE COLLECTION

Watch it grow, one `push_back` at a time

Step into the loop and open your IDE's variable/locals view - watching `friends` gain an element on each iteration is the clearest way to see a `std::vector` actually grow.